Deliverable 4: Stack Justification

Why these tools, and why not C3.ai or Palantir Foundry

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Stack Justification

What was used, and why

Python, pandas, scikit-learn for the data generator and detector. The hard part of this assignment is proving detection works on a realistic failure signature, not building ML infrastructure. LocalOutlierFactor was picked after an actual bake off against IsolationForest, OneClassSVM, and EllipticEnvelope (`notebooks/model_comparison.ipynb`), not by reasoning alone.

Streamlit first, then React and TypeScript with a FastAPI backend. Streamlit validated the detection approach in an afternoon. Once it was proven (11 out of 11 detected, a real evaluated threshold tradeoff), the interface was rebuilt properly, since a shift supervisor's actual tool needs to feel trustworthy, not like a script re-running top to bottom on every click.

A fast, low-cost LLM for the explanation and fleet copilot layers. Constrained explicitly to the numbers it is handed, never inventing a reading or a timeframe, with a deterministic fallback if the call fails so the console never breaks because of a missing key or a network blip.

Playwright and GitHub Actions instead of Tosca, K6, or Azure DevOps. No access to Nucor's actual tooling, so the closest accessible equivalents, built to mirror the same shape a real pipeline would have: generate data, train, evaluate, test, gate on all of it.

Why not C3.ai or Palantir Foundry

For a production version, standardizing on whichever platform Nucor already runs is probably the right call. For this prototype, it would have been the wrong one.

Both platforms front load real modeling investment, C3.ai's Type and Blueprint system, Foundry's ontology layer, that pays off when integrating dozens of data sources across an enterprise under one governed model. For a one week exercise where the actual question is whether one specific anomaly detection approach works on one specific failure mode, that ceremony would have eaten most of the week before a single line of detection logic got written.

C3 AI Reliability specifically already sells this exact problem category, packaged. Building on top of it would mean testing C3.ai's model and C3.ai's assumptions about failure signatures, not the ones actually reasoned through here, which defeats the point of an assignment scoring judgment about the problem, not the ability to configure a vendor product.

There was no access to either platform, which made this partly moot, but the same call would stand with access: prove the idea cheap and fast first, then re-platform onto the governed enterprise system once it is worth the integration investment, not before the underlying approach is even known to work.

If this proved out, the real version becomes a registered model inside whichever platform Nucor standardizes on: data ingestion through that platform's own historian integration, the model sitting in its governance layer (access control, audit, retraining triggers), and the supervisor surface embedded in existing control room tooling rather than a standalone web app.